

Energy Pulse Logging & Compression Scheme

(Authoritative Design Note)

1. Measurement Model

- Each **pulse** represents a fixed quantum of energy (Wh per pulse).
 - Pulse timing encodes **instantaneous power**.
 - Original raw data format:
 - (epoch_seconds, DeltaT_microseconds)
 - Energy is **counted pulses**.
 - Power is **derived from timing**, not directly measured.
-

2. Design Goals

1. Preserve **exact energy** (lossless)
 2. Preserve **high-resolution power derivation**
 3. Reduce **storage size**
 4. Eliminate **duplicate epoch issues**
 5. Enable **fast Grafana queries**
 6. Maintain **continuous, non-resetting history**
-

3. Conceptual Shift

The data stream is **continuous**.

There are:

- no semantic “runs”
- no resets
- no logical boundaries

Compression introduces **storage blocks**, which are purely an implementation detail.

4. Storage Block Format (Binary Concept)

Each block represents a compressed span of pulses:

Field	Size	Description
start_epoch	4 bytes	Unix epoch (seconds) of first pulse
duration_ms	2 bytes	Elapsed time of block (milliseconds)
pulse_count	1 byte	Number of pulses in block

Total: 7 bytes per block

- Little-endian
 - No padding
 - No alignment assumptions
-

5. Physical Validity Ranges

- start_epoch (uint32): valid through year 2106
 - duration_ms (uint16): 0–65,535 ms
 - pulse_count (uint8): 0–255 pulses
(255 pulses/sec $\approx$ 918 kW $\rightarrow$ physically impossible)
-

6. Derived Quantities

Given a block:

(start_epoch, duration_ms, pulse_count)

Energy (exact)

Energy = pulse_count $\times$ Wh_per_pulse

Average Power over block

Watts = (pulse_count $\times$ 3,600,000) / duration_ms

Power precision error $\leq$ 0.5 ms / duration

$\rightarrow$ <0.5% worst case, typically far better

7. Block Accumulation Algorithm (Formal)

State Variables

current_start_epoch
current_duration_ms
current_pulse_count

Initialization

On first pulse:

```
current_start_epoch = pulse_epoch  
current_duration_ms = 0  
current_pulse_count = 1  
last_pulse_time_us = pulse_time_us
```

For each subsequent pulse:

1. Compute time delta from previous pulse:

- $\text{delta_us} = \text{pulse_time_us} - \text{last_pulse_time_us}$

• Tentatively accumulate:

```
new_duration_ms = current_duration_ms + round(delta_us / 1000)  
new_pulse_count = current_pulse_count + 1
```

2. If any limit would be exceeded:

- $\text{new_duration_ms} > 65535$
- $\text{new_pulse_count} > 255$

Then:

- Emit current block
- Start new block with this pulse

3. Else:

- Commit accumulation
- Update state

4. Update last_pulse_time_us

Finalization

At end of data:

- Emit current block
-

8. Block Boundaries

Block boundaries are caused **only** by:

- numeric overflow limits
- implementation convenience

They **do not** imply:

- load stability
 - semantic segmentation
 - resets of any kind
-

9. Database Semantics

- Stored data is **absolute and continuous**
 - No cumulative totals stored
 - No daily resets stored
 - No synthetic IDs required
 - Epoch values are unique per block
-

10. Visualization & Query Semantics (Grafana)

- **Cumulative kWh:**
 - Computed in query
 - Offset to zero at start of time range
- **Daily kWh:**
 - Computed via per-day windowing
 - Resets at midnight (query logic)
- **Instantaneous / Average Power:**
 - Derived from `pulse_count / duration_ms`

All resets occur **at query time**, never in storage.

11. Key Invariants

- Energy is **strictly conserved**
 - Compression is **lossless**
 - Power remains **high-resolution**
 - Storage is **space-efficient**
 - Time ordering is **unambiguous**
-

12. Design Summary (One Sentence)

This system stores continuous pulse-derived energy data as compressed time blocks that preserve exact energy and near-original power resolution, while deferring all resets, accumulation, and visualization semantics to query time.

Final block rules (implemented exactly)

Raw input

Each pulse:

- epoch (seconds)
- `DeltaT` (μs since previous pulse)

Glitch rule

- Ignore pulse if `DeltaT` < 50_000 μs
- Count and report glitches

Output record (“block”) rules

1. Low-rate case ($\Delta T \geq 1$ second)

- Emit **one output record per pulse**
- No aggregation
- Fields:
 - `run_start` = epoch
 - `duration_ms` = `DeltaT` / 1000
 - `pulse_count` = 1

2. High-rate case (multiple pulses per second)

- Aggregate pulses into a block
- Block closes when:
 - elapsed time ≥ 1000 ms **OR**
 - pulse_count reaches 255
- Block fields:
 - run_start = epoch of first pulse in block
 - duration_ms = $\text{sum}(\text{DeltaT}) / 1000$
 - pulse_count = number of pulses

3. Continuity

- Block state is preserved across:
 - file boundaries
 - directories
 - years
- Only flushed:
 - at end of full ingest
 - or when block-closing conditions are met